

Antes de começar

Preparação completa do ambiente de desenvolvimento para construir aplicações Flask prontas para produção.



Ferramentas Necessárias

Python 3.11+

```
python3 --version
```

Linguagem base para todo o desenvolvimento

Git

```
git --version
```

Controle de versão essencial

Docker

```
docker --version
```

Containerização para deploy

Editor de Código

VS Code recomendado

Ambiente de desenvolvimento integrado

 **AVISO:** SQLite é usado para simplificar nos primeiros exercícios. Para implantações em produção, o PostgreSQL é fortemente recomendado.

Ferramentas Comumente Usadas

Python 3.11+

Versão 3.12 funciona bem. Base de toda a aplicação.

Flask 3.0+

Framework web minimalista e poderoso.

SQLAlchemy 2.0+

ORM com SQLite (fácil de trocar por PostgreSQL).

Alembic

Sistema de migrações de banco de dados.

pytest

Framework de testes automatizados.

Docker e Docker Compose

Containerização e orquestração.

Gunicorn

Servidor WSGI para produção.

Nginx

Proxy reverso de alta performance.

Git

Controle de versão distribuído.

O Que Você Terá ao Final da Parte 1

01

Estrutura de projeto limpa

Fábrica de aplicativos, blueprints, configuração organizada

02

Modelos de banco de dados

Com sistema completo de migrações

03

Sistema de autenticação

Registro de usuários, login, logout e redefinição de senha

04

Templates consistentes

Layout unificado em todas as páginas

05

Páginas de erro profissionais

Que ajudam os usuários em vez de confundi-los

Fundamentos

Os Quatro Pilares da Prontidão para Produção



Pilar 1: Confiabilidade

O sistema continua funcionando

Mesmo quando algo dá errado

Exemplo prático:

Se o banco ficar indisponível por alguns segundos, a aplicação trata o erro, registra o ocorrido e responde ao usuário sem quebrar o sistema.

Elementos essenciais:

- Tratamento de erros
- Recuperação graciosa de falhas
- Integridade dos dados
- Reinício automático
- Verificações de saúde (*health checks*)

Pilar 2: Segurança

Proteção total

Somente usuários autorizados acessam recursos

Exemplo prático:

As senhas são armazenadas com hash, as requisições usam HTTPS, e tentativas repetidas de acesso são bloqueadas automaticamente.

Elementos essenciais:

- Hash de senhas
- Proteção contra CSRF (usuário **logado** em um sistema é induzido, sem perceber, a executar uma ação indesejada)
- Defesa contra SQL Injection (atacante consegue **injetar código SQL malicioso** em uma entrada da aplicação e o banco executa esse código **como se fosse legítimo**)
- HTTPS / TLS
- Limitação de requisições (*rate limiting* - limita **quantas requisições** um cliente pode fazer a um sistema **em um intervalo de tempo**)

Pilar 3: Observabilidade

Visibilidade total

É possível entender o que está acontecendo em tempo real

Exemplo prático:

Quando uma requisição falha, o sistema registra logs estruturados, identifica onde ocorreu o erro e permite monitorar métricas e alertas.

Elementos essenciais:

- Logs estruturados
- Rastreamento de requisições
- Monitoramento de erros
- Coleta de métricas
- Alertas

Pilar 4: Manutenibilidade

Evolução contínua

O sistema pode evoluir sem gerar caos

Exemplo prático:

Uma nova funcionalidade é adicionada com testes automatizados, migração de banco versionada e deploy (implantação) contínuo sem interromper o serviço.

Elementos essenciais:

- Testes automatizados
- Migrações de banco de dados
- Separação de configurações
- Pipeline de CI/CD (**fluxo automatizado** que leva o código **do commit até a produção**, com testes, validações e deploy feitos **sem improviso humano**)
- Documentação

Uma aplicação pronta para produção não é a que apenas funciona, mas a que **continua funcionando**, pode ser **monitorada, protegida e evoluída**.

O Que "Produção" Realmente Significa

Seu ambiente de desenvolvimento é indulgente. A produção não é.

Em desenvolvimento:

- Um usuário (você)
- Erros mostram páginas de depuração úteis
- O banco de dados está no seu laptop
- Reiniciar leva segundos
- Nada depende do seu app funcionar

Em produção:

- Múltiplos usuários simultâneos
- Erros não devem vaziar informações sensíveis
- O banco de dados está em um servidor que precisa de backups
- O tempo de inatividade afeta pessoas reais
- Você não pode simplesmente "testar coisas" para ver se funcionam

"Pronto para produção" significa que seu app pode sobreviver a condições do mundo real: Tráfego, falhas, ataques, e você estar dormindo quando algo quebra.

Os Quatro Pilares Explicados

Todo sistema de produção precisa de quatro coisas:

1

Confiabilidade

O aplicativo funciona quando os usuários precisam. Isso significa:

- Lidar com erros de forma elegante em vez de travar
- Recuperar de problemas de conexão com banco de dados
- Não perder dados quando algo dá errado
- Ter uma forma de reiniciar sem intervenção manual

2

Segurança

O aplicativo protege a si mesmo e seus usuários. Isso significa:

- Hash das senhas corretamente
- Proteção contra ataques comuns (CSRF, XSS, SQL injection)
- Não expor dados sensíveis em logs ou mensagens de erro
- Usar HTTPS para todo o tráfego
- Manter as dependências atualizadas

3

Observabilidade

Quando algo dá errado, você pode descobrir o quê. Isso significa:

- Logs estruturados com IDs de requisição
- Rastreamento de erros que alerta
- Métricas que mostram a saúde do sistema
- Capacidade de rastrear um problema do sintoma até a causa

4

Manutenibilidade

Você (ou outra pessoa) pode mudar o app com segurança. Isso significa:

- Testes automatizados que detectam regressões
- Migrações de banco de dados que podem ser aplicadas sem tempo de inatividade
- Configuração separada do código
- Um processo de implantação repetível

Modos de Falha Comuns

Essas são as formas pelas quais os aplicativos Flask normalmente falham em produção.

Desastres de Configuração



Segredos expostos

Segredos codificados fixamente que acabam no Git



Modo debug ativo

Modo depuração ativado em produção
(vazamentos de rastreios de pilha)



Configurações inconsistentes

Configurações diferentes entre local e produção

Problemas com Banco de Dados

Sem migrações

Mudanças de esquema exigem SQL manual

N+1 consultas

Funcionam localmente mas expiram com dados reais

Sem backups

Perda de dados é permanente

Exaustão de conexão

Sistema trava sob carga

Vulnerabilidades de Segurança

Senhas em texto simples

Armazenadas sem hash adequado

Sem proteção CSRF

Formulários vulneráveis a ataques

Injeção SQL

Por concatenação de strings

Dados sensíveis em logs

Informações críticas expostas

Pesadelos de Implantação

Servidor de desenvolvimento em produção

O servidor de desenvolvimento do Flask:

- Vem **embutido** no Flask
- Existe **só para desenvolvimento local**
- Foi feito para testar rápido, não para operar sistema
- Servidor **não é otimizado** para carga
- **Não escala** (1 processo, 1 thread)
- Pode vaziar variáveis

📌 **Nunca use o servidor de desenvolvimento em produção!** Use Gunicorn, uWSGI ou outro servidor WSGI adequado.

Sem Estratégia de Rollback

O problema

Deploy é feito e:

- Quebra login
- Derruba API
- Corrompe dados

E ninguém sabe voltar atrás.

O que é correto



Deploy versionado

Possibilidade de voltar para a versão anterior

Por que isso é crítico

Produção não é laboratório.

Sem rollback:

- Downtime prolongado
- Pressão em cima do time
- Correção às pressas → mais bugs



Banco e aplicação pensados juntos

Migrações compatíveis com rollback

Migrações que Quebram Dados

O erro clássico

Alterar estrutura de tabela sem estratégia

Exemplo:

- Remover coluna usada
- Mudar tipo de dado
- Tornar campo NOT NULL com dados nulos existentes

O que é correto

01

Migrações incrementais e reversíveis

03

Backups antes da migração

Por que isso é devastador

- Perda irreversível de dados
- Sistema sobe, mas dados estão errados
- Bug silencioso (o pior tipo)

02

Testadas antes em staging e base com dados reais (anonimizados)

04

Script de rollback de banco

Seu Ambiente de Desenvolvimento

Certifique-se de ter:

Python 3.11+

```
python3 --version
```

ou `py --version` no Windows

Git

```
git --version
```

Editor de código

VS Code recomendado

Terminal

Com o qual você se sinta confortável

Instalar Python e Configurar no VS Code

Passo a passo limpo

Vamos configurar o ambiente de desenvolvimento completo, começando pela instalação do Python no Windows.

ETAPA 1 — Instalar o Python no Windows

👉 Isso vem antes de qualquer coisa no VS Code.

01

Acesse:
<https://www.python.org/downloads/windows/>

02

Baixe o **Python 3.x (64-bit)**

03

Execute o instalador e **MARQUE OBRIGATORIAMENTE:**

- **Add Python to PATH**
- **Install launcher for all users** (se aparecer)

04

Clique em **Install Now**

05

Ao terminar, **feche e abra o VS Code novamente**



Teste da Instalação do Python

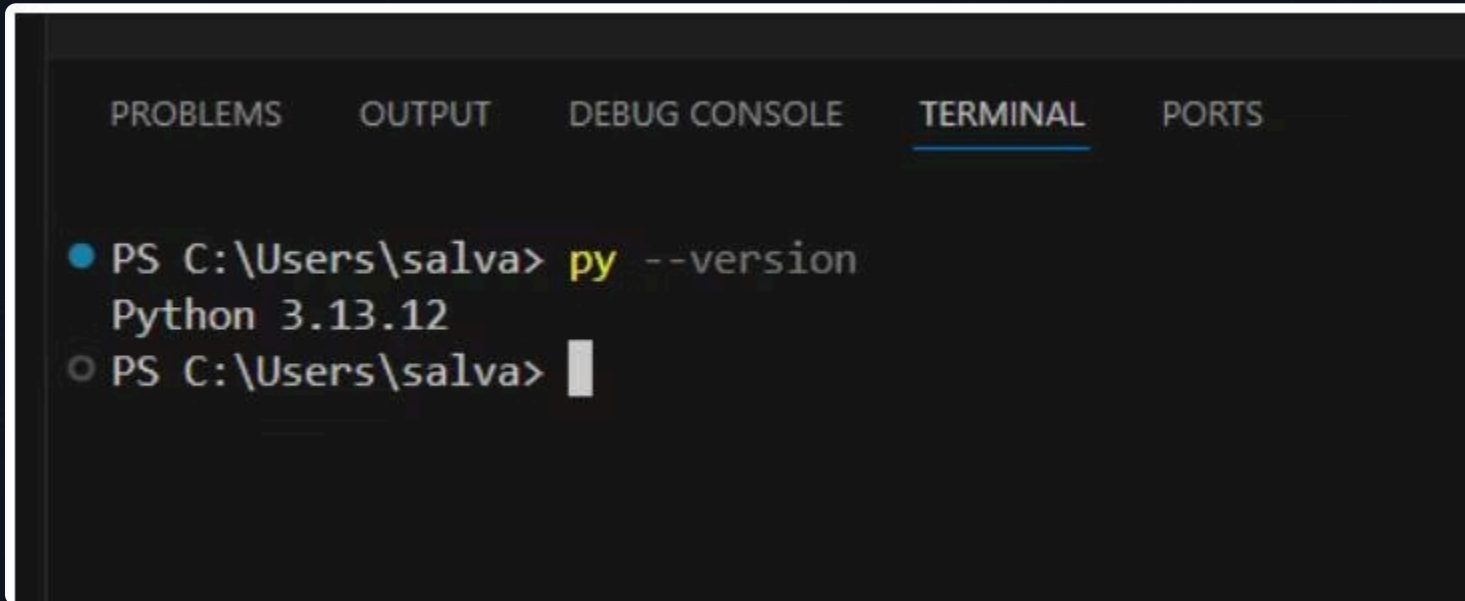
Teste no PowerShell ou Terminal do VS Code:

```
python --version
```

ou

```
py --version
```

- ✓ Se aparecer Python 3.x, siga adiante
- ☒ Se não aparecer, **pare aqui** e corrija a instalação



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
```

```
● PS C:\Users\salva> py --version
Python 3.13.12
○ PS C:\Users\salva> 
```

ETAPA 2 — Abrir o VS Code do Jeito Certo

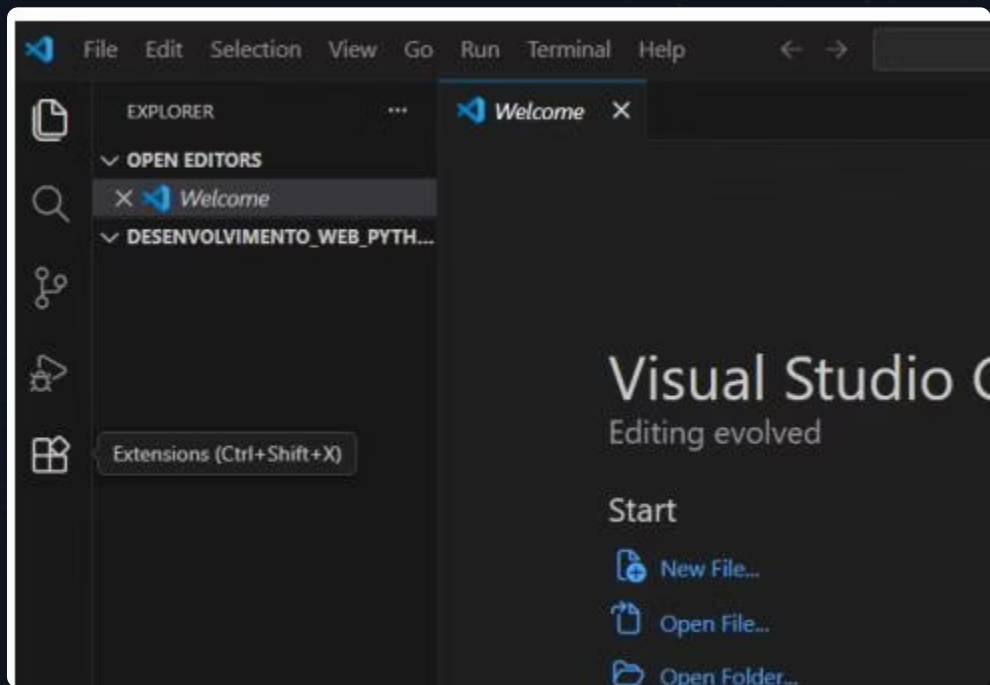
1 Crie ou selecione uma pasta (Windows Explorer), por exemplo: `Desenvolvimento_Web_Python`

2 Abra o **VS Code**

3 Menu **File** → **Open Folder**

4 Selecione a pasta, por exemplo: `Desenvolvimento_Web_Python`

❖ **Nunca comece com arquivo solto. Sempre com pasta aberta.**



ETAPA 3 — Instalar a Extensão Python

01

No VS Code, clique em **Extensions**

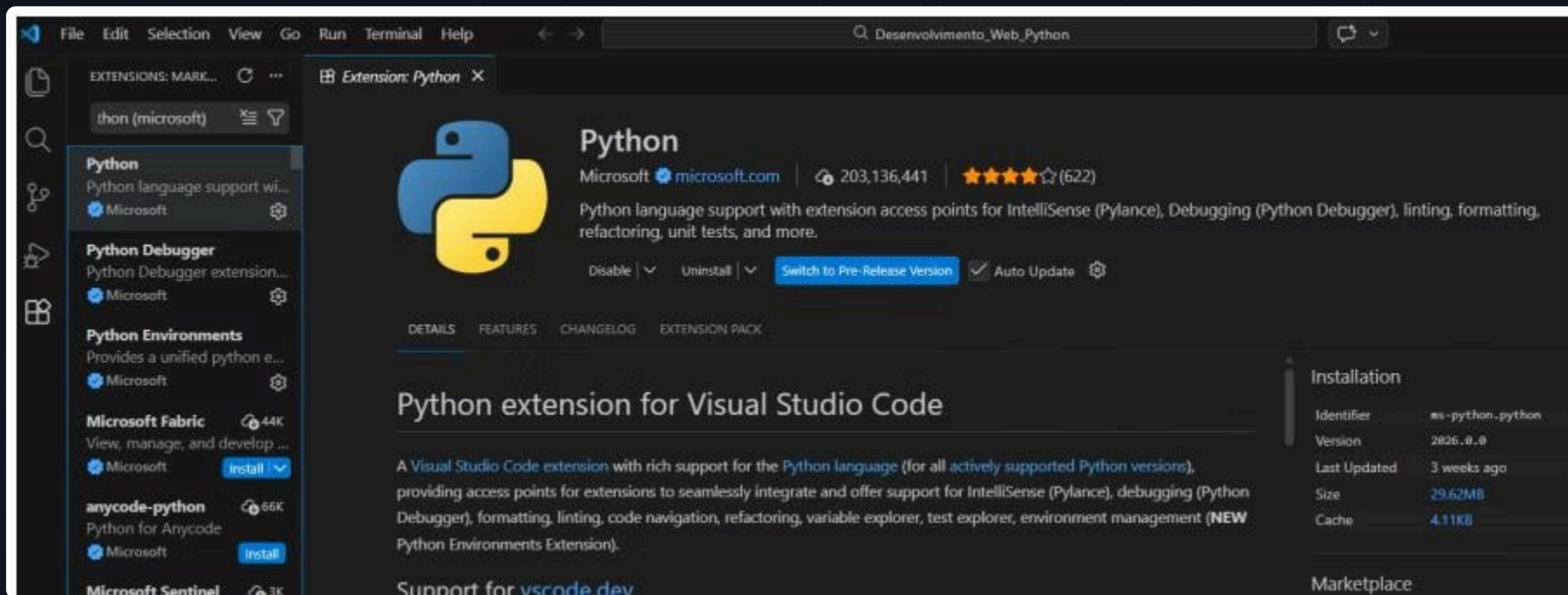
02

Procure por **Python (Microsoft)**

03

Instale

Sem essa extensão, nada funciona.



ETAPA 4 — Criar o Ambiente Virtual (venv)

01

Pressione: **Ctrl + Shift + P** (pode acontecer de precisar usar o **Ctrl + Shift** do lado direito do teclado)

03

Pressione **Enter**

05

Escolha **Venv**

02

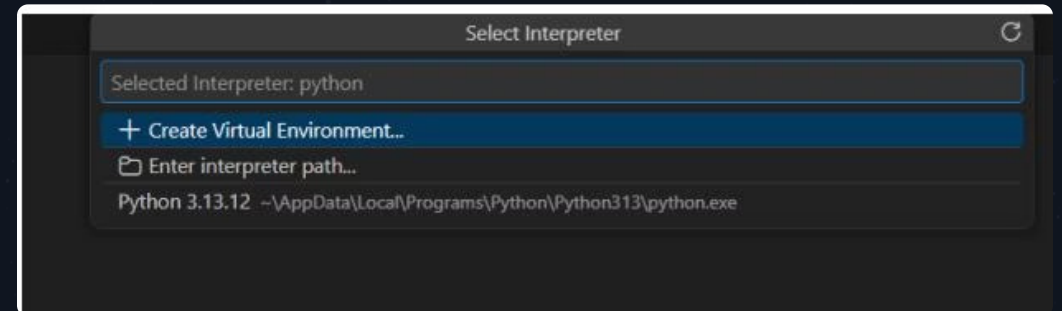
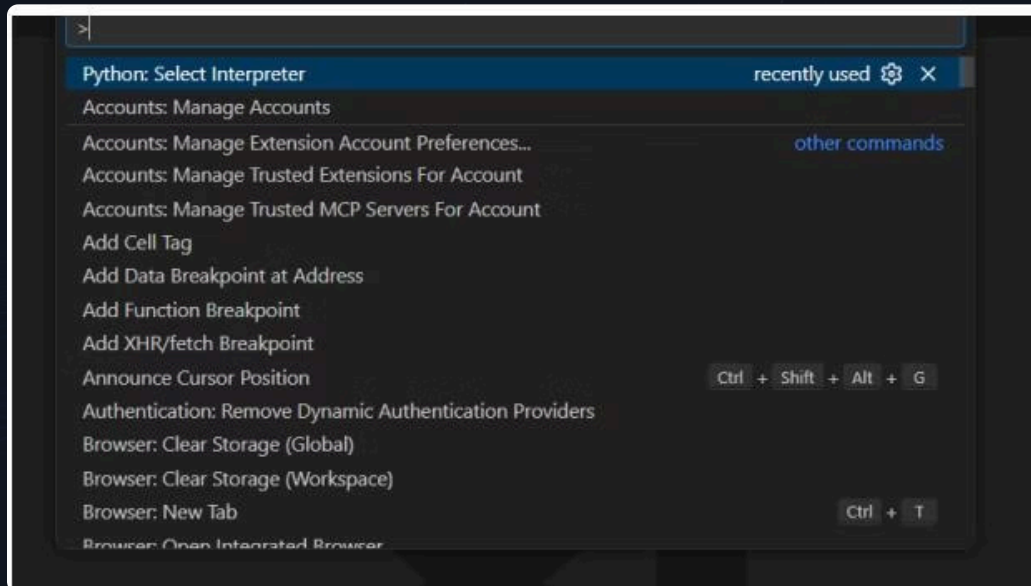
Digite ou selecione: **Python: Select Interpreter**

04

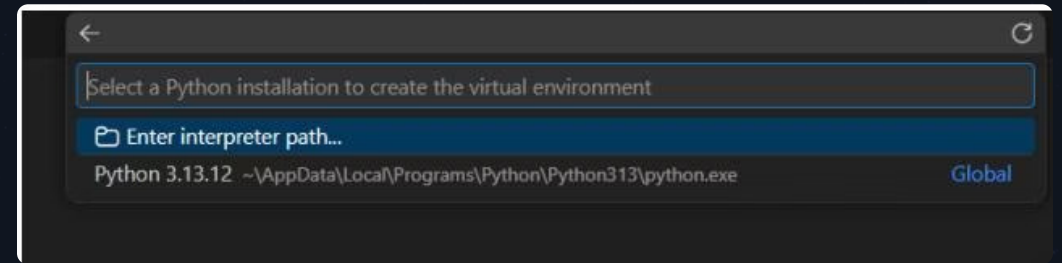
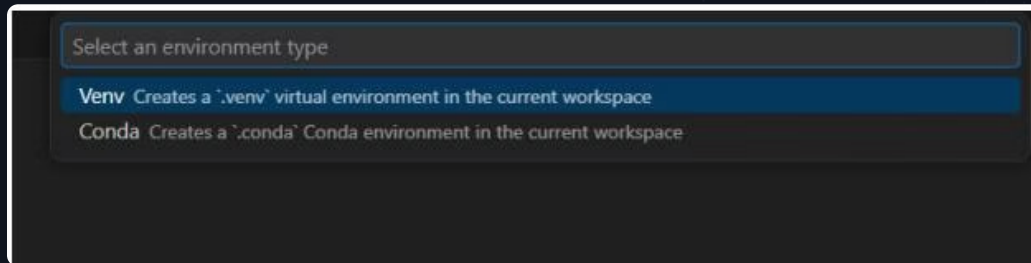
Clique em **Create Virtual Environment...**

06

Selecione o Python sugerido (3.x)



Criando o Ambiente Virtual (continuação)



- ✓ O VS Code vai criar a pasta `.venv` automaticamente
- ✓ O ambiente será ativado

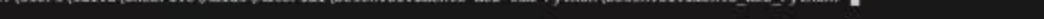
ETAPA 5 — Conferir se Está Tudo Certo

Verifique no canto inferior direito:

Python 3.x (.venv)

Ou no terminal:

```
python --version
```



The screenshot shows a PowerShell terminal window with the following commands and output:

```
PS C:\Users\salsa\OneDrive\Aulas\Material\Desenvolvimento Web com Python\Desenvolvimento_Web_Python> & "C:/Users/salsa/OneDrive/Aulas/Material/Desenvolvimento Web com Python/Desenvolvimento_Web_Python/.venv/Scripts/Activate.ps1"
(.venv) PS C:\Users\salsa\OneDrive\Aulas\Material\Desenvolvimento Web com Python\Desenvolvimento_Web_Python> python --version
Python 3.13.12
(.venv) PS C:\Users\salsa\OneDrive\Aulas\Material\Desenvolvimento Web com Python\Desenvolvimento_Web_Python>
```

ETAPA 6 — Teste Final (Primeiro Código)

1

Crie um arquivo: `teste.py`

2

Escreva: `print("Python funcionando")`

3

Execute: `python teste.py`

Se aparecer a mensagem → acabou. Ambiente pronto.

Git

Verifique se o Git está instalado:

```
git --version
```

```
(.venv) PS C:\Users\salva\OneDrive\Aulas\Material\Desenvolvimento Web com Python\Desenvolvimento_Web_Python> git --version  
git version 2.53.0.windows.1  
(.venv) PS C:\Users\salva\OneDrive\Aulas\Material\Desenvolvimento Web com Python\Desenvolvimento_Web_Python> |
```

Docker

Docker, de forma simples, é uma **ferramenta que empacota uma aplicação junto com tudo o que ela precisa para funcionar** — sistema, dependências e configurações — dentro de um **container**.

Esse container roda da mesma forma em qualquer computador, evitando o clássico "funciona na minha máquina".

Diferente do ambiente virtual do Python, que isola apenas bibliotecas da linguagem, o Docker isola **tudo o ambiente de execução da aplicação**, tornando a implantação, o teste e a distribuição de sistemas mais previsíveis e padronizados.

É muito usado quando a aplicação precisa rodar igual em desenvolvimento, homologação e produção.

📄 **Necessário depois, mas instale agora**

Instalando o Docker

01

Instalar **Docker Desktop for Windows**

<https://docs.docker.com/desktop/setup/install/windows-install/>

03

Reboot

02

Durante a instalação, marque:

- **Use WSL 2 instead of Hyper-V (recommended)**
- Deixe **Windows Containers** desmarcado

04

Verificar: `docker --version`

Por Que WSL 2?

WSL 2 (Recomendado)

Docker roda de forma mais leve, moderna e compatível com Python, Linux e backend

👉 **Resultado:** Docker preparado para backend moderno (Python, APIs, banco, etc.)

Windows Containers

Não é para aula, nem para Python, é para cenários corporativos específicos (.NET legado)

```
docker --version
```

Preparação

MISSÃO 0 — "Seu App Vai Quebrar"

"Hoje vocês vão descobrir por que 90% dos apps caseiros nunca deveriam ir para produção."



FASE 1 — O Caos (15 min)

Se você subir um app Flask hoje e divulgar no Instagram, o que pode dar errado nas próximas 24 horas?

Respostas em grupo. Por exemplo:

- Alguém invade
- Banco trava
- App cai
- Dados somem
- Dois usuários editam ao mesmo tempo
- Senhas expostas
- Erro 500 sem explicação

Agora você organiza isso nos 4 pilares:

Problema	Pilar
App cai	Confiabilidade
Invasão	Segurança
Não sei por que deu erro	Observabilidade
Código impossível de entender	Manutenibilidade



FASE 2 — "Contrato de Produção" (10 min)

Se esse app fosse seu negócio, quanto tempo ele pode ficar fora do ar?

99%

Disponibilidade básica

~7 horas por mês fora

99.9%

Disponibilidade boa

~43 minutos por mês

99.99%

Disponibilidade excelente

~4 minutos por mês

Missão:

Cada grupo escreve uma frase de SLA.

Um SLA básico tem 4 elementos:

1 O que está sendo garantido

2 Quanto

3 Em qual período

4 O que acontece se não cumprir

(opcional, mas profissional)

Isso já coloca mentalidade profissional no primeiro dia.

FASE FINAL — Gancho para Próxima Aula

O que é mais importante: escrever rápido
ou escrever certo?

Semana que vem vamos criar o primeiro endpoint.

📌 **Endpoint** é um endereço da sua API onde o cliente envia uma requisição para executar uma ação específica.

Próximos Passos

Ambiente configurado. Conceitos fundamentais compreendidos. Agora é hora de começar a construir aplicações Flask prontas para produção.

